

Tugas Besar II IF3270 Pembelajaran Mesin Convolutional Neural Network dan Recurrent Neural Network



Oleh :

Muhammad Ghifary Komara Putra 13523066

Muh. Rusmin Nurwadin 13523068

Aryo Wisanggeni 13523100

**Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026**

Daftar Isi

Daftar Isi	1
Daftar Gambar	2
Daftar Tabel	3
I. Deskripsi Persoalan	4
II. Penjelasan Implementasi	5
2.1 Deskripsi Umum	5
2.2 Deskripsi Kelas Implementasi	5
2.2.1 CNN	6
2.2.2 RNN dan LSTM	9
2.3 Deskripsi Forward Propagation	17
2.3.1 CNN	17
2.3.2 RNN	20
2.3.3 LSTM	21
III. Hasil Pengujian	24
3.1 CNN	24
3.1.1 Perbandingan Shared vs Non-Shared Parameter	25
3.1.2 Pengaruh jumlah layer konvolusi	26
3.1.3 Pengaruh banyak filter per layer konvolusi	27
3.1.4 Pengaruh ukuran filter per layer konvolusi	27
3.1.5 Pengaruh jenis pooling layer	28
3.1.6 Perbandingan Keras vs from scratch	28
3.2 RNN dan LSTM	29
3.2.1 Perbandingan Jumlah Layer (RNN)	30
3.2.2 Perbandingan Jumlah Layer (LSTM)	31
3.2.3 Perbandingan Ukuran hidden state (RNN)	32
3.2.4 Perbandingan Ukuran hidden state (LSTM)	33
3.2.5 Perbandingan RNN vs LSTM	34
3.2.6 Perbandingan Keras vs from scratch	40
3.2.7 Pengaruh panjang maksimum caption terhadap BLEU-4	41
IV. Kesimpulan dan Saran	42
4.1 Kesimpulan	42
4.2 Saran	42
V. Pembagian Tugas	43
Referensi	44
Lampiran	45

Daftar Gambar

Gambar 1. Eksperimen Variasi Arsitektur CNN Keras.....	25
Gambar 2. Shared vs Non-Shared Loss Comparison.....	26
Gambar 3. Pengaruh jumlah layer konvolusi pada 8 pasangan konfigurasi.....	27
Gambar 4. Pengaruh banyak filter per layer konvolusi pada 8 pasangan konfigurasi.....	28
Gambar 5. Pengaruh ukuran filter per layer konvolusi pada 8 pasangan konfigurasi.....	28
Gambar 6. Pengaruh jenis pooling layer pada 8 pasangan konfigurasi.....	29
Gambar 7. Eksperimen Variasi Arsitektur RNN dan LSTM Keras BLEU-4.....	30
Gambar 8. Eksperimen Variasi Arsitektur RNN dan LSTM Keras METEOR.....	30
Gambar 9. Nilai BLEU-4 dan METEOR pada Eksperimen Variasi Recurrent Layer RNN.....	31
Gambar 10. Grafik Loss dan Validation Loss pada Eksperimen Variasi Recurrent Layer RNN...	32
Gambar 11. Nilai BLEU-4 dan METEOR pada Eksperimen Variasi Recurrent Layer LSTM.....	32
Gambar 12. Grafik Loss dan Validation Loss pada Eksperimen Variasi Recurrent Layer LSTM	33
Gambar 13. Nilai BLEU-4 dan METEOR pada Eksperimen Variasi Hidden Size RNN.....	33
Gambar 14. Grafik Loss dan Validation Loss pada Eksperimen Variasi Hidden Size RNN.....	34
Gambar 15. Nilai BLEU-4 dan METEOR pada Eksperimen Variasi Hidden Size LSTM.....	34
Gambar 16. Grafik Loss dan Validation Loss pada Eksperimen Variasi Recurrent Layer LSTM	35
Gambar 17. Perbandingan Rata-Rata Waktu Eksekusi, Nilai BLEU-4, METEOR pada RNN dan LSTM.....	35
Gambar 18. Perbandingan Grafik Loss dan Validation Loss pada RNN dan LSTM.....	36
Gambar 19. Perbandingan Keras vs from scratch.....	41
Gambar 20. Grafik Hubungan Panjang Maksimum Caption dengan Nilai BLEU-4.....	42

Daftar Tabel

Tabel 1. Detail Konfigurasi Model CNN dan Hasil Evaluasi.....	25
Tabel 2. Hasil Perbandingan Model Shared Parameter dengan Non-Shared Parameter.....	26
Tabel 3. Hasil Perbandingan Model Shared Parameter Keras vs From-Scratch.....	29
Tabel 4. Hasil Perbandingan Model RNN Keras.....	31
Tabel 5. Hasil Perbandingan Model LSTM Keras.....	31
Tabel 6. Tabel Perbandingan Caption Hasil RNN dan LSTM.....	36

I. Deskripsi Persoalan

Tugas Besar II IF3270 Pembelajaran Mesin menugaskan peserta kuliah untuk mengimplementasikan pustaka *Convolutional Neural Network* (CCNN) dan *Recurrent Neural Network* (RNN) dari awal, untuk kemudian dibandingkan dengan pustaka umum dalam menyelesaikan masalah tertentu. Implementasi mencakup *forward* dan *backward propagation* pada CNN, *Simple RNN*, dan LSTM. Lebih lanjut, akan dilakukan eksplorasi terhadap arsitektur *encoder-decoder* yang menggabungkan RNN CNN dan LSTM.

Bagian CNN diimplementasikan dalam konteks *image classification* menggunakan dataset Intel Image Classification. Kemudian, bagian RNN dan LSTM diimplementasikan dalam konteks *image captioning* menggunakan dataset Flickr8k dengan arsitektur Show and Tell. Kedua modul dilatih secara terpisah sebagai *encoder* dan *decoder*. Dataset Intel Image Classification berisi data gambar dalam *folder* sesuai dengan klasifikasinya masing-masing, yaitu *buildings*, *forest*, *glacier*, *mountain*, *sea*, dan *street*. Di sisi lain, dataset Flickr8k berisi pasangan *image* dan *text* yang merepresentasikan *caption*/deskripsi dari *image* tersebut.

Setelah pustaka CNN dan RNN telah diimplementasikan, begitu pula dengan model terkait, akan dilakukan pengujian untuk memastikan validitas hasil implementasi tersebut, juga sebagai bentuk eksplorasi pengaruh arsitektur terhadap performa model. Secara umum, skema pengujian adalah sebagai berikut:

1. CNN
 - Perbandingan shared vs non-shared parameter
 - Pengaruh jumlah layer konvolusi
 - Pengaruh banyak filter per layer konvolusi
 - Pengaruh ukuran filter per layer konvolusi
 - Pengaruh jenis pooling layer
 - Perbandingan implementasi Keras vs *from scratch*
2. RNN dan LSTM
 - Perbandingan jumlah layer (RNN)
 - Perbandingan jumlah layer (LSTM)
 - Perbandingan ukuran *hidden state* (RNN)
 - Perbandingan ukuran *hidden state* (LSTM)
 - Perbandingan RNN vs LSTM
 - Perbandingan implementasi Keras vs *from scratch*
 - Pengaruh panjang maksimum *caption* terhadap BLEU-4

II. Penjelasan Implementasi

2.1 Deskripsi Umum

Implementasi modul CNN, RNN, LSTM, dan eksperimen disusun dalam susunan direktori sebagai berikut.



2.2 Deskripsi Kelas Implementasi

Sebagai catatan, penjelasan modul terkait *auto differentiation* dan FFNN tidak disertakan karena sudah terdapat pada laporan Tugas Besar I.

2.2.1 CNN

- Conv2D

```
# Shared Parameter
class Conv2D(Module):
    def __init__(self, in_channels, out_channels, kernel_size, stride=1,
padding=0):
        super().__init__()
        self.kernel_size = kernel_size if isinstance(kernel_size, tuple) else
(kernel_size, kernel_size)
        self.stride = stride
        self.padding = padding

        self.weight = None # Shape: (kH, kW, C_in, C_out)
        self.bias = None # Shape: (C_out,)

    def forward(self, x):
        # x shape: (N, H, W, C_in)
        if isinstance(x, Value): x = x.data

        N, H, W, C_in = x.shape
        kH, kW = self.kernel_size
        out_H = (H + 2*self.padding - kH) // self.stride + 1
        out_W = (W + 2*self.padding - kW) // self.stride + 1

        if self.padding > 0:
            x = np.pad(x, ((0,0), (self.padding, self.padding), (self.padding,
self.padding), (0,0)), mode='constant')

            output = np.zeros((N, out_H, out_W, self.weight.shape[-1]))

            for i in range(out_H):
                for j in range(out_W):
                    h_start, w_start = i * self.stride, j * self.stride
                    patch = x[:, h_start:h_start+kH, w_start:w_start+kW, :] # (N,
kH, kW, C_in)
                    output[:, i, j, :] = np.tensordot(patch, self.weight,
axes=((1, 2, 3), (0, 1, 2))) + self.bias

            return output
```

Kelas ini merepresentasikan *convolutional layer encoder* CNN yang akan dikembangkan. Atribut pada kelas ini merepresentasikan informasi yang diperlukan dan akan disimpan oleh *layer*, yaitu *kernel_size*, *stride*, *padding*, *weight*, dan *bias*. Method *forward* akan melakukan konvolusi sesuai parameter/atribut pada kelas bersangkutan, dengan menggunakan *shared parameter*.

- LocallyConnected2D

```
# Non-shared Parameter
class LocallyConnected2D(Module):
    def __init__(self, in_channels, out_channels, input_size, kernel_size,
stride=1):
        super().__init__()
        self.kernel_size = kernel_size if isinstance(kernel_size, tuple) else
(kernel_size, kernel_size)
        self.stride = stride
        self.input_size = input_size # (H, W)

        # Shape: (out_H * out_W, kH * kW * C_in, C_out)
        self.weight = None
        self.bias = None # Shape: (out_H, out_W, C_out)

    def forward(self, x):
        N, H, W, C_in = x.shape
        kH, kW = self.kernel_size
        out_H = (H - kH) // self.stride + 1
        out_W = (W - kW) // self.stride + 1
        C_out = self.bias.shape[-1]

        output = np.zeros((N, out_H, out_W, C_out))
        # Non-shared parameters: weight depends on the spatial position (i, j)
        for i in range(out_H):
            for j in range(out_W):
                h_s, w_s = i * self.stride, j * self.stride
                patch = x[:, h_s:h_s+kH, w_s:w_s+kW, :].reshape(N, -1)

                # Weight index for position (i, j)
                idx = i * out_W + j
                output[:, i, j, :] = (patch @ self.weight[idx]) + self.bias[i,
j]

        return output
```

Kelas ini merepresentasikan *convolutional layer encoder* CNN yang akan dikembangkan. Atribut pada kelas ini merepresentasikan informasi yang diperlukan dan akan disimpan oleh *layer*, yaitu *kernel_size*, *stride*, *padding*, *weight*, dan *bias*. Method *forward* akan melakukan konvolusi sesuai parameter/atribut pada kelas bersangkutan, tanpa menggunakan *shared parameter*.

- MaxPooling2D dan AveragePooling2D

```
# Sliding window pooling
class MaxPooling2D(Module):
    def __init__(self, pool_size=(2, 2), stride=2):
        super().__init__()
        self.pool_size = pool_size
        self.stride = stride

    def forward(self, x):
```



```

        N, H, W, C = x.shape
        kH, kW = self.pool_size
        out_H = (H - kH) // self.stride + 1
        out_W = (W - kW) // self.stride + 1

        output = np.zeros((N, out_H, out_W, C))
        for i in range(out_H):
            for j in range(out_W):
                h_s, w_s = i * self.stride, j * self.stride
                output[:, i, j, :] = np.max(x[:, h_s:h_s+kH, w_s:w_s+kW, :],
axis=(1, 2))
            return output

class AveragePooling2D(Module):
    def __init__(self, pool_size=(2, 2), stride=2):
        super().__init__()
        self.pool_size = pool_size
        self.stride = stride

    def forward(self, x):
        N, H, W, C = x.shape
        kH, kW = self.pool_size
        out_H = (H - kH) // self.stride + 1
        out_W = (W - kW) // self.stride + 1

        output = np.zeros((N, out_H, out_W, C))
        for i in range(out_H):
            for j in range(out_W):
                h_s, w_s = i * self.stride, j * self.stride
                output[:, i, j, :] = np.mean(x[:, h_s:h_s+kH, w_s:w_s+kW, :],
axis=(1, 2))
            return output

```

Kelas ini merepresentasikan *pooling layer encoder* CNN yang akan dikembangkan. Atribut pada kelas ini merepresentasikan informasi yang diperlukan dan akan disimpan oleh *layer*, yaitu *pool_size* dan *stride*. Method *forward* akan melakukan *pooling* sesuai parameter/atribut pada kelas bersangkutan, sesuai dengan nama kelas masing-masing (MaxPooling atau AveragePooling) dengan menggunakan *sliding window*.

- GlobalAveragePooling2D dan GlobalMaxPooling2D

```

# Global Pooling
class GlobalAveragePooling2D(Module):
    def forward(self, x):
        return np.mean(x, axis=(1, 2))

class GlobalMaxPooling2D(Module):
    def forward(self, x):
        return np.max(x, axis=(1, 2))

```

Kelas ini merepresentasikan *pooling layer encoder* CNN yang akan dikembangkan. Atribut pada kelas ini merepresentasikan informasi yang diperlukan dan akan disimpan oleh *layer*, yaitu *pool_size* dan *stride*. Method *forward* akan melakukan *pooling* sesuai parameter/atribut pada kelas bersangkutan, sesuai dengan nama kelas masing-masing (MaxPooling atau AveragePooling) dengan menggunakan *global pooling* tanpa *sliding window*.

- Flatten

```
class Flatten(Module):
    def forward(self, x):
        return x.reshape(x.shape[0], -1)
```

Kelas ini merepresentasikan proses *flattening tensor* menjadi vektor satu dimensi untuk menjembatani CNN dengan arsitektur *neural network* lainnya (*dense layer*, *recurrent layer*, dan sebagainya).

- Lainnya

Selain modul yang disebutkan, *file-file* lain pada repositori berisi fungsi-fungsi bantuan yang digunakan untuk keperluan eksperimen.

2.2.2 RNN dan LSTM

Sebagai catatan, deskripsi kelas pada subbab ini dilakukan menurut sesuai keterurutan alfabet.

- Dense ProjectionLayer dan DenseOutputLayer

```
class DenseProjectionLayer(Linear):
    def forward(self, x):
        return super().forward(x)

    def load_keras(self, weights):
        self.load_state(dense_state(weights))

class DenseOutputLayer(Linear):
    def forward(self, x):
        return super().forward(x).softmax(axis=-1)

    def load_keras(self, weights):
        self.load_state(dense_state(weights))
```

Kelas ini merupakan *wrapper* dari *dense layer* untuk menyamakan dengan konvensi penamaan dari arsitektur RNN yang akan dikembangkan, dengan penyesuaian *forward propagation* dengan fungsi aktivasi sesuai konvensi pada spesifikasi.

- Embedding

```
class Embedding(Module):
    def __init__(self, vocab_size, embedding_dim):
        super().__init__()

        self.vocab_size = vocab_size
        self.embedding_dim = embedding_dim
        self.weight = Value(
            np.random.randn(vocab_size, embedding_dim) * 0.01,
            label="embedding_weight"
        )

    def forward(self, indices):
        out = Value(
            self.weight.data[indices],
            (self.weight,),
            'embedding'
        )

        def _backward():
            grad = np.zeros_like(self.weight.data)
            np.add.at(grad, indices, out.grad)
            self.weight.grad += grad
            out._backward = _backward

        return out

    def parameters(self):
        return [self.weight]

    def state_dict(self):
        return {
            "weight": np.array(self.weight.data, copy=True),
        }

    def load_state(self, state):
        weight = np.array(state["weight"], dtype=float, copy=True)
        if weight.shape != self.weight.data.shape:
            raise ValueError

        self.weight.data = weight
        self.weight.grad = np.zeros_like(self.weight.data)

    def load_keras(self, weights):
        self.load_state({"weight": weights[0]})

EmbeddingLayer = Embedding
```

Kelas ini merepresentasikan dari *embedding layer* pada arsitektur *decoder* yang akan dikembangkan. Kelas ini digunakan untuk memetakan token ID menjadi *embedding vector*. *Vocab_size* menyatakan banyak token (unik) dalam *vocabulary*, *embedding_dim* menyatakan dimensi setiap *embedding vector*, dan *weight* menyatakan bobot pada *layer*. *Method-method* yang disediakan oleh kelas ini serupa dengan turunan kelas *Module* lainnya, yaitu untuk keperluan *forward propagation*, *backward propagation*, serta melakukan *load* terhadap bobot.

- LSTMCell dan LSTM

```
class LSTMCell(Module):
    def __init__(self, input_size, hidden_size):
        super().__init__()

        self.hidden_size = hidden_size
        def init_gate(in_dim):
            return Value(
                np.random.randn(in_dim, hidden_size)
                / np.sqrt(in_dim)
            )

        self.W_f = init_gate(input_size)
        self.U_f = init_gate(hidden_size)
        self.b_f = Value(np.zeros((1, hidden_size)))

        self.W_i = init_gate(input_size)
        self.U_i = init_gate(hidden_size)
        self.b_i = Value(np.zeros((1, hidden_size)))

        self.W_c = init_gate(input_size)
        self.U_c = init_gate(hidden_size)
        self.b_c = Value(np.zeros((1, hidden_size)))

        self.W_o = init_gate(input_size)
        self.U_o = init_gate(hidden_size)
        self.b_o = Value(np.zeros((1, hidden_size)))

    def forward(self, x_t, h_prev, c_prev):
        f_t = (
            x_t @ self.W_f
            + h_prev @ self.U_f
            + self.b_f
        ).sigmoid()

        i_t = (
            x_t @ self.W_i
            + h_prev @ self.U_i
            + self.b_i
        ).sigmoid()

        c_tilde = (
            x_t @ self.W_c
            + h_prev @ self.U_c
            + self.b_c
```

```

        ).tanh()

    o_t = (
        x_t @ self.W_o
        + h_prev @ self.U_o
        + self.b_o
    ).sigmoid()

    c_t = f_t * c_prev + i_t * c_tilde

    h_t = o_t * c_t.tanh()

    return h_t, c_t

def parameters(self):
    return [
        self.W_f, self.U_f, self.b_f,
        self.W_i, self.U_i, self.b_i,
        self.W_c, self.U_c, self.b_c,
        self.W_o, self.U_o, self.b_o,
    ]

def state_dict(self):
    return {
        "W_f": np.array(self.W_f.data, copy=True),
        "U_f": np.array(self.U_f.data, copy=True),
        "b_f": np.array(self.b_f.data, copy=True),
        "W_i": np.array(self.W_i.data, copy=True),
        "U_i": np.array(self.U_i.data, copy=True),
        "b_i": np.array(self.b_i.data, copy=True),
        "W_c": np.array(self.W_c.data, copy=True),
        "U_c": np.array(self.U_c.data, copy=True),
        "b_c": np.array(self.b_c.data, copy=True),
        "W_o": np.array(self.W_o.data, copy=True),
        "U_o": np.array(self.U_o.data, copy=True),
        "b_o": np.array(self.b_o.data, copy=True),
    }

def load_state(self, state):
    for name in self.state_dict():
        if name not in state:
            raise ValueError

    for name in self.state_dict():
        param = getattr(self, name)
        param.data = np.array(state[name], dtype=float, copy=True)
        param.grad = np.zeros_like(param.data)

def load_keras(self, weights):
    if len(weights) < 3:
        raise ValueError

    kernel = np.array(weights[0], dtype=float, copy=True)
    recurrent = np.array(weights[1], dtype=float, copy=True)
    bias = np.array(weights[2], dtype=float, copy=True).reshape(-1)
    hidden = self.hidden_size

```

```

        if kernel.shape[1] != 4 * hidden:
            raise ValueError

    self.load_state({
        "W_i": kernel[:, 0:hidden],
        "W_f": kernel[:, hidden:2 * hidden],
        "W_c": kernel[:, 2 * hidden:3 * hidden],
        "W_o": kernel[:, 3 * hidden:4 * hidden],
        "U_i": recurrent[:, 0:hidden],
        "U_f": recurrent[:, hidden:2 * hidden],
        "U_c": recurrent[:, 2 * hidden:3 * hidden],
        "U_o": recurrent[:, 3 * hidden:4 * hidden],
        "b_i": bias[0:hidden].reshape(1, hidden),
        "b_f": bias[hidden:2 * hidden].reshape(1, hidden),
        "b_c": bias[2 * hidden:3 * hidden].reshape(1, hidden),
        "b_o": bias[3 * hidden:4 * hidden].reshape(1, hidden),
    })

class LSTM(Module):
    def __init__(self, input_size, hidden_size, layers=1):
        super().__init__()

        self.hidden_size = hidden_size
        self.layers = int(layers)
        if self.layers <= 0:
            raise ValueError

        self.cells = []
        in_size = input_size
        for _ in range(self.layers):
            cell = LSTMCell(in_size, hidden_size)
            self.cells.append(cell)
            in_size = hidden_size

        self.cell = self.cells[0]

    def children(self):
        return list(self.cells)

    def forward(self, x, h0=None, c0=None):
        x = x if isinstance(x, Value) else Value(x)
        batch_size, seq_len, _ = x.data.shape
        layer_input = x
        last_h = None
        last_c = None

        for index, cell in enumerate(self.cells):
            if h0 is None:
                h_t = Value(np.zeros((batch_size, self.hidden_size)))
            elif isinstance(h0, (list, tuple)):
                h_t = h0[index]
            else:
                h_t = h0

            if c0 is None:

```

```

        c_t = Value(np.zeros((batch_size, self.hidden_size)))
    elif isinstance(c0, (list, tuple)):
        c_t = c0[index]
    else:
        c_t = c0

    outputs = []
    for t in range(seq_len):
        x_t = layer_input[:, t, :]
        h_t, c_t = cell(
            x_t,
            h_t,
            c_t
        )
        outputs.append(h_t)

    layer_input = Value.stack(outputs, axis=1)
    last_h = h_t
    last_c = c_t

    return layer_input, (last_h, last_c)

def parameters(self):
    params = []
    for cell in self.cells:
        params.extend(cell.parameters())
    return params

def state_dict(self):
    return [cell.state_dict() for cell in self.cells]

def load_state(self, states):
    if len(states) != len(self.cells):
        raise ValueError
    for cell, state in zip(self.cells, states):
        cell.load_state(state)

def load_keras(self, weights):
    if len(weights) != self.layers:
        raise ValueError
    for cell, cell_weights in zip(self.cells, weights):
        cell.load_keras(cell_weights)

```

Kedua kelas ini merepresentasikan *layer* LSTM pada *decoder* yang akan dikembangkan. Atribut *hidden_size* digunakan untuk menentukan dimensi *layer*, lalu terdapat pula atribut yang menyimpan bobot untuk *gate-gate* pada LSTM. Kemudian, *method* yang tersedia serupa dengan turunan kelas *Module* lainnya, dengan penyesuaian pada bagian *forward propagation* mengikuti proses perhitungan dan interaksi antar-*gate* pada LSTM.

- SimpleRNNCell dan RNN

```
class SimpleRNNCell(Module):
    def __init__(self, input_size, hidden_size):
        super().__init__()

        self.input_size = input_size
        self.hidden_size = hidden_size
        self.W_xh = Value(
            np.random.randn(input_size, hidden_size)
            / np.sqrt(input_size),
            label="W_xh"
        )

        self.W_hh = Value(
            np.random.randn(hidden_size, hidden_size)
            / np.sqrt(hidden_size),
            label="W_hh"
        )

        self.b_h = Value(
            np.zeros((1, hidden_size)),
            label="b_h"
        )

    def forward(self, x_t, h_prev):
        h_t = (
            x_t @ self.W_xh
            + h_prev @ self.W_hh
            + self.b_h
        ).tanh()

        return h_t

    def parameters(self):
        return [
            self.W_xh,
            self.W_hh,
            self.b_h
        ]

    def state_dict(self):
        return {
            "W_xh": np.array(self.W_xh.data, copy=True),
            "W_hh": np.array(self.W_hh.data, copy=True),
            "b_h": np.array(self.b_h.data, copy=True),
        }

    def load_state(self, state):
        for name in ("W_xh", "W_hh", "b_h"):
            if name not in state:
                raise ValueError

        self.W_xh.data = np.array(state["W_xh"], dtype=float, copy=True)
        self.W_hh.data = np.array(state["W_hh"], dtype=float, copy=True)
```



```

        self.b_h.data = np.array(state["b_h"], dtype=float, copy=True)

        self.W_xh.grad = np.zeros_like(self.W_xh.data)
        self.W_hh.grad = np.zeros_like(self.W_hh.data)
        self.b_h.grad = np.zeros_like(self.b_h.data)

    def load_keras(self, weights):
        if len(weights) < 3:
            raise ValueError
        self.load_state({
            "W_xh": weights[0],
            "W_hh": weights[1],
            "b_h": weights[2],
        })

class RNN(Module):
    def __init__(self, input_size, hidden_size, layers=1):
        super().__init__()

        self.hidden_size = hidden_size
        self.layers = int(layers)
        if self.layers <= 0:
            raise ValueError

        self.cells = []
        in_size = input_size
        for _ in range(self.layers):
            cell = SimpleRNNCell(in_size, hidden_size)
            self.cells.append(cell)
            in_size = hidden_size

        self.cell = self.cells[0]

    def children(self):
        return list(self.cells)

    def forward(self, x, h0=None):
        x = x if isinstance(x, Value) else Value(x)
        batch_size, seq_len, _ = x.data.shape
        layer_input = x
        last_hidden = None

        for index, cell in enumerate(self.cells):
            if h0 is None:
                h_t = Value(np.zeros((batch_size, self.hidden_size)))
            elif isinstance(h0, (list, tuple)):
                h_t = h0[index]
            else:
                h_t = h0

            outputs = []
            for t in range(seq_len):
                x_t = layer_input[:, t, :]
                h_t = cell(x_t, h_t)
                outputs.append(h_t)

```

```

        layer_input = Value.stack(outputs, axis=1)
        last_hidden = h_t

    return layer_input, last_hidden

def parameters(self):
    params = []
    for cell in self.cells:
        params.extend(cell.parameters())
    return params

def state_dict(self):
    return [cell.state_dict() for cell in self.cells]

def load_state(self, states):
    if len(states) != len(self.cells):
        raise ValueError
    for cell, state in zip(self.cells, states):
        cell.load_state(state)

def load_keras(self, weights):
    if len(weights) != self.layers:
        raise ValueError
    for cell, cell_weights in zip(self.cells, weights):
        cell.load_keras(cell_weights)

```

Kedua kelas ini merepresentasikan *layer* RNN pada *decoder* yang akan dikembangkan. Atribut *input_size* digunakan untuk menentukan dimensi *feature vector*, *hidden_size* digunakan untuk menentukan dimensi *layer*, lalu terdapat pula atribut yang menyimpan bobot pada *layer* ini. Kemudian, *method* yang tersedia serupa dengan turunan kelas *Module* lainnya, dengan penyesuaian pada bagian *forward propagation* mengikuti proses perhitungan pada RNN.

- Lainnya

Selain modul yang disebutkan, *file-file* lain pada repositori berisi fungsi-fungsi bantuan yang digunakan untuk keperluan eksperimen.

2.3 Deskripsi *Forward Propagation*

2.3.1 CNN

Forward propagation pada modul CNN melibatkan *method forward* pada beberapa kelas, yaitu Conv2D, LocallyConnected2D, MaxPooling2D, AveragePooling2D, GlobalAveragePooling2D, GlobalMaxPooling2D, dan Flatten.

```
# Conv2D
```

```

def forward(self, x):
    if isinstance(x, Value): x = x.data

    N, H, W, C_in = x.shape
    kH, kW = self.kernel_size
    out_H = (H + 2*self.padding - kH) // self.stride + 1
    out_W = (W + 2*self.padding - kW) // self.stride + 1

    if self.padding > 0:
        x = np.pad(x, ((0,0), (self.padding, self.padding), (self.padding,
self.padding), (0,0)), mode='constant')

    output = np.zeros((N, out_H, out_W, self.weight.shape[-1]))

    for i in range(out_H):
        for j in range(out_W):
            h_start, w_start = i * self.stride, j * self.stride
            patch = x[:, h_start:h_start+kH, w_start:w_start+kW, :]
            output[:, i, j, :] = np.tensordot(patch, self.weight,
axes=((1, 2, 3), (0, 1, 2))) + self.bias

    return output

# LocallyConnected2D
def forward(self, x):
    if isinstance(x, Value): x = x.data

    N, H, W, C_in = x.shape
    kH, kW = self.kernel_size
    out_H = (H + 2 * self.padding - kH) // self.stride + 1
    out_W = (W + 2 * self.padding - kW) // self.stride + 1
    C_out = self.bias.shape[-1]

    if self.padding > 0:
        x = np.pad(
            x,
            ((0, 0), (self.padding, self.padding), (self.padding,
self.padding), (0, 0)),
            mode="constant",
        )

    output = np.zeros((N, out_H, out_W, C_out))
    for i in range(out_H):
        for j in range(out_W):
            h_s, w_s = i * self.stride, j * self.stride
            patch = x[:, h_s:h_s+kH, w_s:w_s+kW, :].reshape(N, -1)
            idx = i * out_W + j
            output[:, i, j, :] = (patch @ self.weight[idx]) + self.bias[i,
j]

    return output

# MaxPooling2D
def forward(self, x):
    if isinstance(x, Value): x = x.data
    N, H, W, C = x.shape
    kH, kW = self.pool_size

```

```

        out_H = (H - kH) // self.stride + 1
        out_W = (W - kW) // self.stride + 1

        output = np.zeros((N, out_H, out_W, C))
        for i in range(out_H):
            for j in range(out_W):
                h_s, w_s = i * self.stride, j * self.stride
                output[:, i, j, :] = np.max(x[:, h_s:h_s+kH, w_s:w_s+kW, :],
axis=(1, 2))
            return output

# AveragePooling2D
def forward(self, x):
    N, H, W, C = x.shape
    kH, kW = self.pool_size
    out_H = (H - kH) // self.stride + 1
    out_W = (W - kW) // self.stride + 1

    output = np.zeros((N, out_H, out_W, C))
    for i in range(out_H):
        for j in range(out_W):
            h_s, w_s = i * self.stride, j * self.stride
            output[:, i, j, :] = np.mean(x[:, h_s:h_s+kH, w_s:w_s+kW, :],
axis=(1, 2))
        return output

# GlobalAveragePooling2D
def forward(self, x):
    return np.mean(x, axis=(1, 2))

# GlobalMaxPooling2d
def forward(self, x):
    return np.max(x, axis=(1, 2))

# Flatten
def forward(self, x):
    if isinstance(x, Value): x = x.data
    return x.reshape(x.shape[0], -1)

```

Pada Conv2D, *forward propagation* dilakukan dengan menggeser *kernel* konvolusi pada seluruh area gambar menggunakan parameter *stride* dan *padding*. Kemudian, berdasarkan parameter tersebut, akan dilakukan menggunakan prinsip *shared parameter*. Di sisi lain, hal serupa dilakukan oleh LocallyConnected2D namun tanpa menggunakan prinsip *shared parameter*.

Selanjutnya, mengenai *pooling layer*, MaxPooling2D dan AveragePooling2D berturut-turut akan melakukan *pooling* berdasarkan nilai maksimum dan rata-rata dengan menggunakan *sliding window* dengan *pool size* dan *stride* yang dapat ditentukan sebelumnya. Jika ingin menggunakan nilai yang sama secara keseluruhan, *pooling layer* dapat memanfaatkan *forwad propagation* pada kelas GlobalMaxPooling2D dan GlobalAveragePooling2D. Kemudian, hasil akhir *forward propagation* CNN adalah proses *flatten* (oleh kelas dengan nama yang sama)

untuk mengubah format tensor menjadi vektor sesuai keperluan persoalan yang akan diselesaikan.

2.3.2 RNN

Forward propagation pada modul RNN melibatkan *method forward* pada dua kelas, yaitu SimpleRNNCell dan RNN. Kelas SimpleRNNCell akan melakukan *forward propagation* pada suatu *time step* sedangkan kelas RNN akan memproses keseluruhan sekuens dengan memanfaatkan *recurrent cell* secara berulang.

```
# SimpleRNNCell
def forward(self, x_t, h_prev):
    h_t = (
        x_t @ self.W_xh
        + h_prev @ self.W_hh
        + self.b_h
    ).tanh()

    return h_t

# RNN
def forward(self, x, h0=None):
    x = x if isinstance(x, Value) else Value(x)
    batch_size, seq_len, _ = x.data.shape
    layer_input = x
    last_hidden = None

    for index, cell in enumerate(self.cells):
        if h0 is None:
            h_t = Value(np.zeros((batch_size, self.hidden_size)))
        elif isinstance(h0, (list, tuple)):
            h_t = h0[index]
        else:
            h_t = h0

        outputs = []
        for t in range(seq_len):
            x_t = layer_input[:, t, :]
            h_t = cell(x_t, h_t)
            outputs.append(h_t)

        layer_input = Value.stack(outputs, axis=1)
        last_hidden = h_t

    return layer_input, last_hidden
```

Kelas SimpleRNNCell akan melakukan perhitungan berikut:

$$h_t = \tanh(x_t W_{xh} + h_{t-1} W_{hh} + b_h)$$

untuk suatu *time step* t . Di sisi lain, pada kelas RNN, method *forward* memanfaatkan SimpleRNNCell untuk memproses seluruh sekuens pada setiap *time step* dengan mempertimbangkan *batch size*, *sequence length*, dan *input dimension*, kemudian melakukan perhitungan berikut :

$$h_t = \tanh(x_t W_{xh} + h_{t-1} W_{hh} + b_h)$$

secara berulang berdasarkan parameter tersebut.

2.3.3 LSTM

Forward propagation pada modul LSTM melibatkan *method forward* pada dua kelas, yaitu LSTMCell dan LSTM. Kelas LSTMCell akan melakukan *forward propagation* pada suatu *time step* dengan memanfaatkan *hidden state* dan *cell state* sebelumnya, sedangkan kelas LSTM akan memproses keseluruhan sekuens dengan memanfaatkan LSTMCell secara berulang.

```
# LSTMCell
def forward(self, x_t, h_prev, c_prev):
    f_t = (
        x_t @ self.W_f
        + h_prev @ self.U_f
        + self.b_f
    ).sigmoid()

    i_t = (
        x_t @ self.W_i
        + h_prev @ self.U_i
        + self.b_i
    ).sigmoid()

    c_tilde = (
        x_t @ self.W_c
        + h_prev @ self.U_c
        + self.b_c
    ).tanh()

    o_t = (
        x_t @ self.W_o
        + h_prev @ self.U_o
        + self.b_o
    ).sigmoid()

    c_t = f_t * c_prev + i_t * c_tilde
```

```

        h_t = o_t * c_t.tanh()

    return h_t, c_t

# LSTM
def forward(self, x, h0=None, c0=None):
    x = x if isinstance(x, Value) else Value(x)
    batch_size, seq_len, _ = x.data.shape
    layer_input = x
    last_h = None
    last_c = None

    for index, cell in enumerate(self.cells):
        if h0 is None:
            h_t = Value(np.zeros((batch_size, self.hidden_size)))
        elif isinstance(h0, (list, tuple)):
            h_t = h0[index]
        else:
            h_t = h0

        if c0 is None:
            c_t = Value(np.zeros((batch_size, self.hidden_size)))
        elif isinstance(c0, (list, tuple)):
            c_t = c0[index]
        else:
            c_t = c0

        outputs = []
        for t in range(seq_len):
            x_t = layer_input[:, t, :]
            h_t, c_t = cell(
                x_t,
                h_t,
                c_t
            )
            outputs.append(h_t)

        layer_input = Value.stack(outputs, axis=1)
        last_h = h_t
        last_c = c_t

    return layer_input, (last_h, last_c)

```

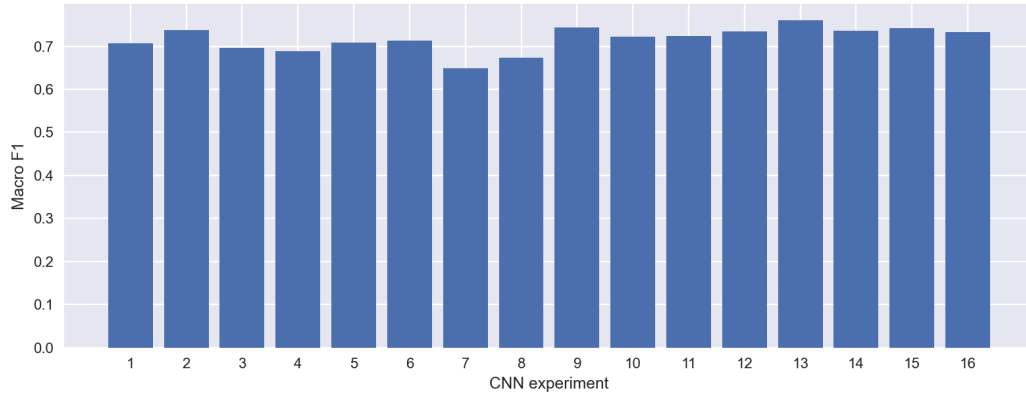
Kelas LSTMCell akan melakukan beberapa perhitungan *gate*, yaitu *forget gate*, *input gate*, *cell state*, dan *output gate* dengan persamaan sebagai berikut:

$$\begin{aligned}
f_t &= \sigma(x_t W_f + h_{t-1} U_f + b_f) \\
i_t &= \sigma(x_t W_i + h_{t-1} U_i + b_i) \\
\tilde{c}_t &= \tanh(x_t W_c + h_{t-1} U_c + b_c) \\
c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \\
o_t &= \sigma(x_t W_o + h_{t-1} U_o + b_o) \\
h_t &= o_t \odot \tanh(c_t)
\end{aligned}$$

Di sisi lain, pada kelas LSTM, *method forward* memanfaatkan LSTMCell untuk memproses seluruh sekuens pada setiap *time step* dengan mempertimbangkan *batch size*, *sequence length*, dan *input dimension*, kemudian melakukan perhitungan seluruh *gate* dan *hidden state* baru secara berulang berdasarkan *parameter* tersebut.

III. Hasil Pengujian

3.1 CNN



Gambar 1. Eksperimen Variasi Arsitektur CNN Keras

Grafik di atas menunjukkan ringkasan umum eksperimen yang telah dilakukan berdasarkan metrik macro *F1-score*, untuk seluruh eksplorasi variasi. Detail konfigurasi setiap variasi dan analisis lebih lanjut akan dipaparkan pada subbab di bawah ini.

Tabel 1. Detail Konfigurasi Model CNN dan Hasil Evaluasi

Konfigurasi	Jumlah Layer	Filter	Kernel	Pooling	Macro F1
1	2	32	3	Max	0.7073
2	2	32	3	Average	0.7371
3	2	32	5	Max	0.6957
4	2	32	5	Average	0.6890
5	2	64	3	Max	0.7087
6	2	64	3	Average	0.7124
7	2	64	5	Max	0.6492
8	2	64	5	Average	0.6737
9	3	32	3	Max	0.7439
10	3	32	3	Average	0.7228
11	3	32	5	Max	0.7233

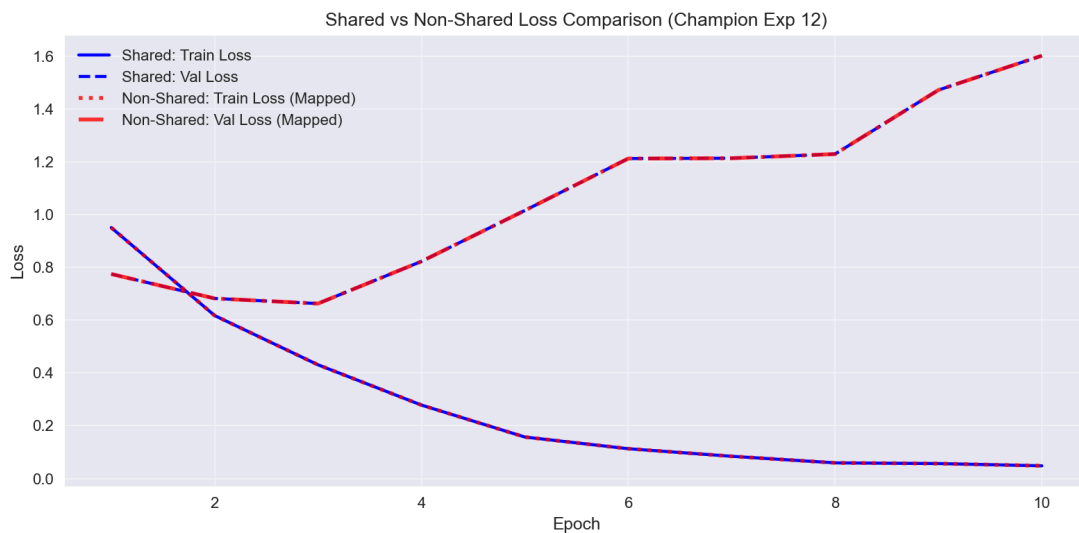
12	3	32	5	Average	0.7340
13	3	64	3	Max	0.7606
14	3	64	3	Average	0.7356
15	3	64	5	Max	0.7418
16	3	64	5	Average	0.7336

3.1.1 Perbandingan *Shared* vs *Non-Shared Parameter*

Dari hasil pengujian 16 konfigurasi model yang ditampilkan pada **tabel 1**, konfigurasi yang digunakan untuk model from-scratch adalah 3 layer, 64 filter (channel), 3 kernel, dan menggunakan max pooling (konfigurasi 13).

Tabel 2. Hasil Perbandingan Model Shared Parameter dengan Non-Shared Parameter

Konfigurasi	Parameter	Macro F1
Shared	601,350	0.7606
Non-Shared	1,079,445,702	0.7606



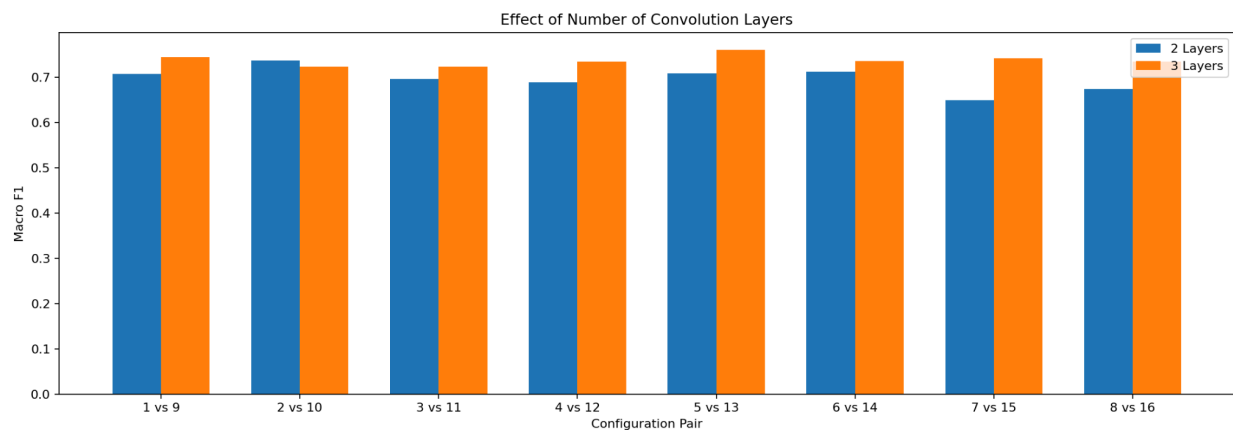
Gambar 2. Shared vs Non-Shared Loss Comparison

Grafik loss pada Eksperimen 12 menunjukkan fenomena **overfitting parah**, *training loss* sukses menurun drastis mendekati nol (~ 0.05), namun *validation loss* melonjak tajam secara eksponensial setelah epoch 3 hingga mencapai nilai ~ 1.60 . Kurva kedua model berimpit secara

identik karena model *non-shared* diinisialisasi menggunakan metode *tiling* (menyalin secara spasial) bobot dari model *shared* yang sudah terlatih. Kemiripan grafik ini membuktikan kebenaran matematis dari alur *forward propagation* lokal, yakni struktur *non-shared* mampu mereplikasi hasil akhir ekstraksi fitur model *shared* selama nilai bobot di setiap koordinatnya disamakan secara seragam.

Meskipun hasil akhirnya sama pada pengujian ini, perbedaan efisiensi arsitektur keduanya sangat berbeda. Model *non-shared* (*LocallyConnected2D*) mengalami **ledakan parameter hingga 1795x lipat** dari model *shared* (Conv2D) pada resolusi 150*150 karena setiap koordinat pixel dipaksa memiliki bobot independen. Jika dilatih dari awal, jumlah parameter raksasa pada model *non-shared* ini akan memicu *overfitting* yang jauh lebih ekstrem.

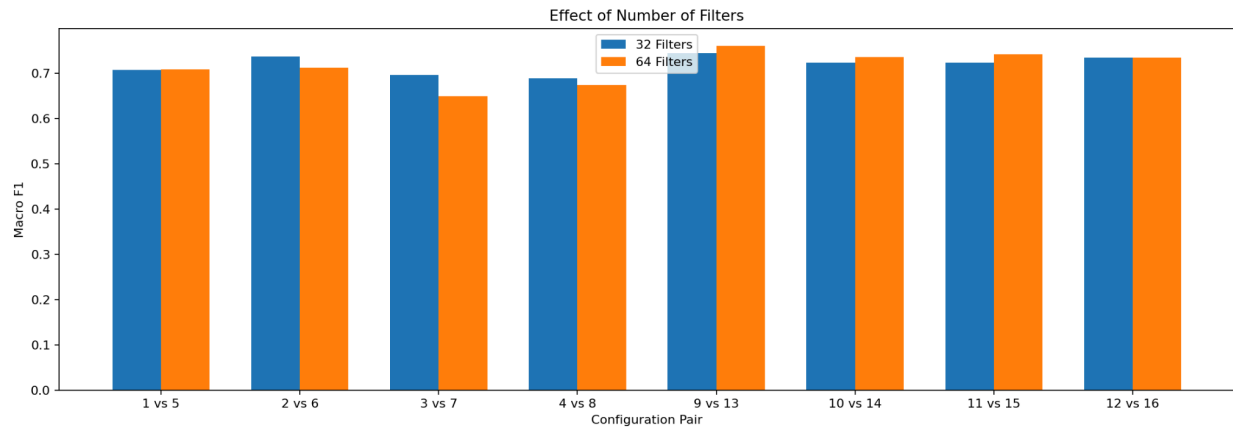
3.1.2 Pengaruh jumlah *layer* konvolusi



Gambar 3. Pengaruh jumlah *layer* konvolusi pada 8 pasangan konfigurasi

Gambar di atas menunjukkan perbandingan jumlah *layer* konvolusi terhadap *macro F1-score* untuk 8 pasangan konfigurasi yang tersedia. Secara umum, model dengan *layer* lebih banyak memiliki performa yang lebih baik. Hanya terdapat satu pasangan konfigurasi dengan pola sebaliknya, yaitu pasangan konfigurasi 2 vs 10. Hal ini terjadi karena *layer* konvolusi yang lebih banyak memungkinkan model CNN untuk mempelajari fitur-fitur yang lebih kompleks dan beragam dari dataset gambar yang tersedia dan memberikan hasil klasifikasi yang lebih baik.

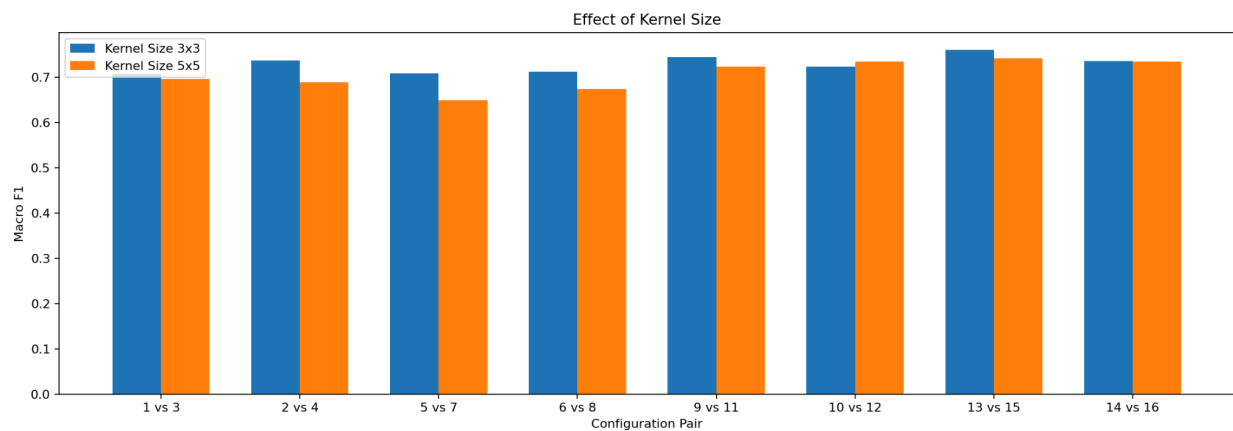
3.1.3 Pengaruh banyak *filter* per *layer* konvolusi



Gambar 4. Pengaruh banyak *filter* per *layer* konvolusi pada 8 pasangan konfigurasi

Gambar di atas menunjukkan perbandingan banyak *filter* konvolusi per *layer* terhadap *macro F1-score* untuk 8 pasangan konfigurasi yang tersedia. Tidak terdapat pola yang begitu terlihat pada eksperimen yang telah dilakukan. Beberapa pasangan konfigurasi memiliki performa yang lebih baik dengan *filter* lebih sedikit, beberapa pasangan lain memiliki performa lebih buruk dengan *filter* lebih sedikit, dan bahkan beberapa pasangan model tidak menunjukkan perbedaan performa yang begitu signifikan. Secara intuitif, *filter* yang lebih banyak seharusnya memungkinkan model untuk menangkap lebih banyak pola pada gambar. Namun, perlu diperhatikan juga bahwa *filter* yang berlebihan (yang juga menandakan banyaknya *parameter*) semakin rentan terhadap *overfitting* atau hanya menambahkan kompleksitas yang tidak perlu pada model yang dikembangkan.

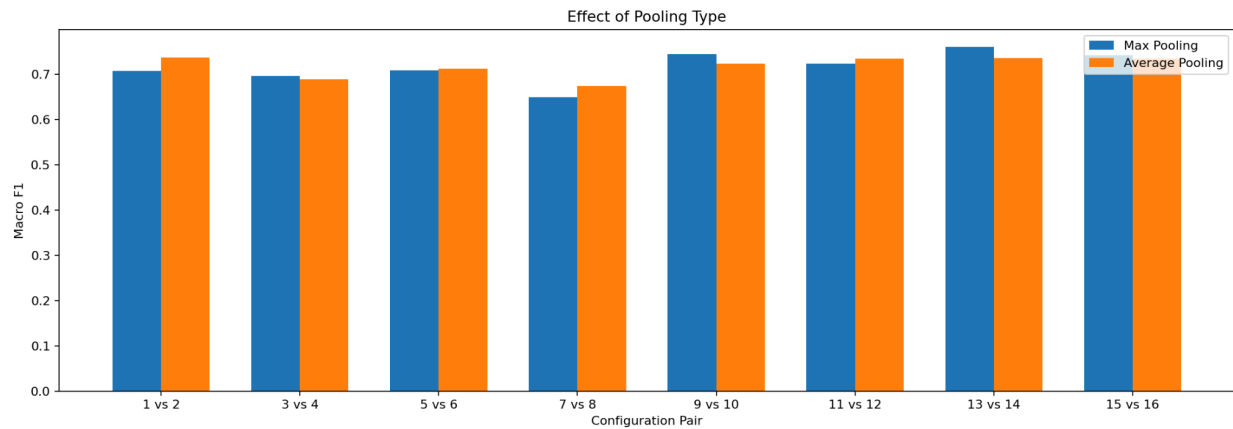
3.1.4 Pengaruh ukuran *filter* per *layer* konvolusi



Gambar 5. Pengaruh ukuran *filter* per *layer* konvolusi pada 8 pasangan konfigurasi

Gambar di atas menunjukkan perbandingan pengaruh ukuran *filter* per *layer* konvolusi terhadap *macro F1-score* untuk 8 pasangan konfigurasi yang tersedia. Secara umum, model dengan ukuran *filter* lebih kecil memiliki performa yang lebih baik. Hanya terdapat satu pasangan konfigurasi dengan pola sebaliknya, yaitu pasangan 10 vs 12. Hal ini terjadi karena dengan ukuran *kernel* yang lebih kecil pola-pola yang ditangkap oleh model akan bersifat lebih *fine-grained*. Selain itu, ukuran *kernel* yang lebih kecil juga membantu mengurangi kompleksitas model dan meningkatkan kemungkinan generalisasi yang lebih baik

3.1.5 Pengaruh jenis *pooling layer*



Gambar 6. Pengaruh jenis *pooling layer* pada 8 pasangan konfigurasi

Gambar di atas menunjukkan perbandingan pengaruh jenis *pooling layer* terhadap *macro F1-score* untuk 8 pasangan konfigurasi yang tersedia. Tidak terdapat pola yang begitu terlihat pada eksperimen yang telah dilakukan. Beberapa pasangan konfigurasi memiliki performa yang lebih baik dengan *max pooling*, beberapa pasangan lain memiliki performa lebih baik dengan *average pooling*, dan bahkan beberapa pasangan model tidak menunjukkan perbedaan performa yang begitu signifikan antar metode *pooling*. Hal ini terjadi karena kedua metode *pooling* memiliki prinsip dan cara kerjanya masing-masing, dan hanya konfigurasi-konfigurasi tertentu saja yang dapat mengambil manfaat lebih dari prinsip *pooling* tersebut.

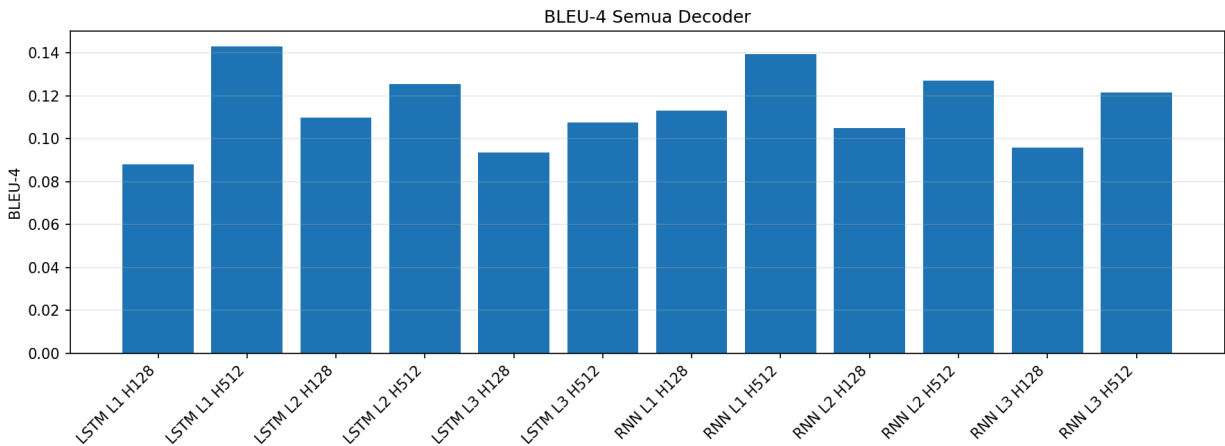
3.1.6 Perbandingan Keras vs *from scratch*

Tabel 3. Hasil Perbandingan Model Shared Parameter Keras vs From-Scratch

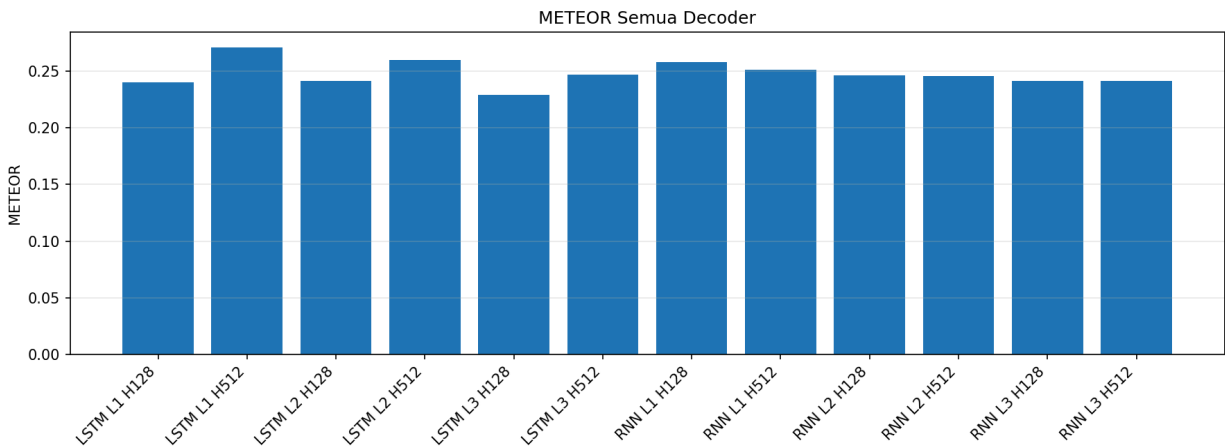
Konfigurasi	Macro F1
Keras	0.7606
From-scratch	0.7606

Berdasarkan pengujian pada data test, hasil *forward propagation from scratch* untuk variasi *shared parameter* (Conv2D) menunjukkan kecocokan persis dengan Keras. Kedua model menghasilkan Macro F1-score yang identik, yaitu 0.7606. Keberhasilan ini dibuktikan secara empiris melalui fungsi `np.allclose(keras_raw, scratch_shared_raw, atol=1e-3)` yang bernilai **True**.

3.2 RNN dan LSTM



Gambar 7. Eksperimen Variasi Arsitektur RNN dan LSTM Keras BLEU-4



Gambar 8. Eksperimen Variasi Arsitektur RNN dan LSTM Keras METEOR

Grafik di atas menunjukkan ringkasan umum eksperimen yang telah dilakukan berdasarkan metrik BLEU-4, untuk seluruh eksplorasi variasi RNN dan LSTM. Detail konfigurasi setiap variasi dan analisis lebih lanjut akan dipaparkan pada subbab di bawah ini.

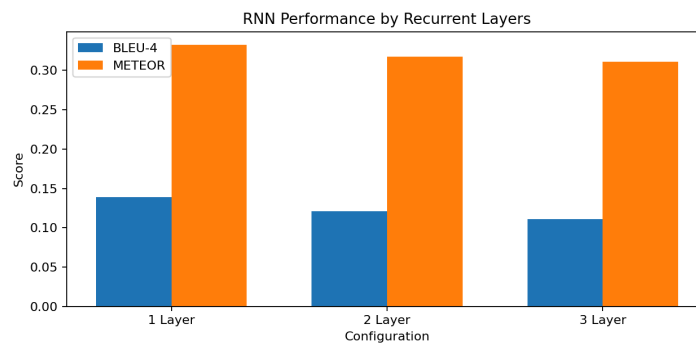
Tabel 4. Hasil Perbandingan Model RNN Keras

Konfigurasi	Layer Recurrent	Hidden State	BLEU-4	METEOR	Waktu Eksekusi
rnn-1x-128	1	128	0.11838	0.317266	45.368093
rnn-2x-128	2	128	0.105427	0.302255	68.292161
rnn-3x-128	3	128	0.089541	0.304962	112.622568
rnn-1x-512	1	512	0.159473	0.347779	42.221393
rnn-2x-512	2	512	0.136186	0.332347	75.877472
rnn-3x-512	3	512	0.132785	0.316641	74.093177

Tabel 5. Hasil Perbandingan Model LSTM Keras

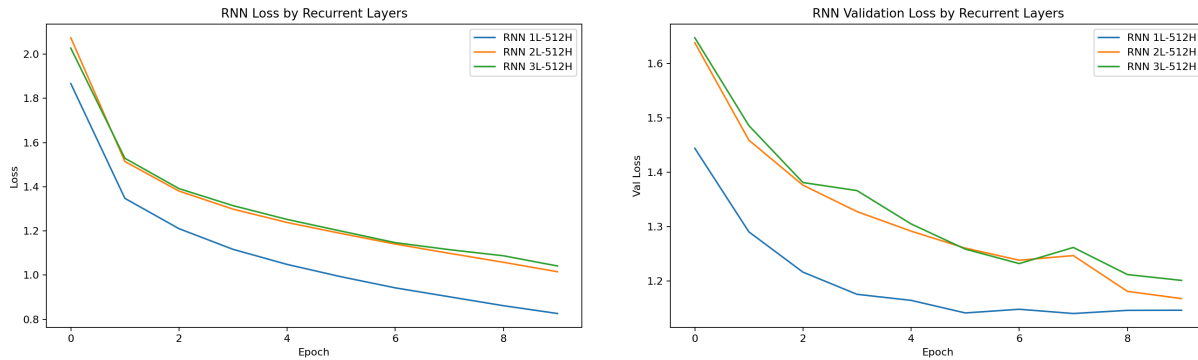
Konfigurasi	Layer Recurrent	Hidden State	BLEU-4	METEOR	Waktu Eksekusi
lstm-1x-128	1	128	0.079523	0.287564	34.820462
lstm-2x-128	2	128	0.111372	0.304746	30.750602
lstm-3x-128	3	128	0.09184	0.282632	21.331626
lstm-1x-512	1	512	0.153895	0.356469	35.994934
lstm-2x-512	2	512	0.132438	0.347233	48.79073
lstm-3x-512	3	512	0.110111	0.321161	47.335904

3.2.1 Perbandingan Jumlah *Layer* (RNN)

**Gambar 9.** Nilai BLEU-4 dan METEOR pada Eksperimen Variasi *Recurrent Layer* RNN

Gambar di atas menunjukkan perbandingan jumlah *layer* pada RNN terhadap nilai BLEU-4, METEOR, disertai dengan grafik *loss* dan *validation loss* pada ilustrasi di bawah paragraf ini. Berdasarkan hasil eksperimen, peningkatan jumlah recurrent layer pada model RNN

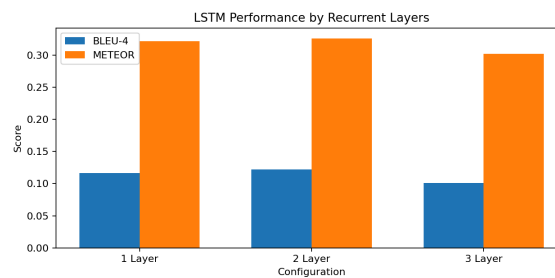
secara umum menurunkan performa caption generation. Dalam kedua metrik tersebut, konfigurasi dengan 1 *layer* menghasilkan performa terbaik. Hal ini menunjukkan bahwa arsitektur RNN yang lebih dalam cenderung mengalami kesulitan optimisasi seperti masalah yang berkaitan dengan *vanishing gradient* sehingga model lebih sulit mempelajari dependensi jangka panjang secara efektif.



Gambar 10. Grafik *Loss* dan *Validation Loss* pada Eksperimen Variasi *Recurrent Layer* RNN

Berdasarkan grafik, model dengan 1 *layer* tetap memberikan performa yang terbaik dan lebih stabil pada *validation loss*. Di sisi lain, kedua model lainnya memiliki pola *loss* yang sama, yaitu terus menurun, namun menjadi semakin tidak stabil pada *validation loss* seiring *epoch* berjalan. Ketidakstabilan ini sangat terlihat pada arsitektur paling kompleks, yaitu 3 *layer*. Hal ini dapat terjadi karena penggunaan arsitektur dengan jumlah *layer* berlebih dapat meningkatkan kompleksitas arsitektur dan meningkatkan tingkat kesulitan arsitektur untuk melakukan generalisasi sepanjang keberjalanan proses *training*.

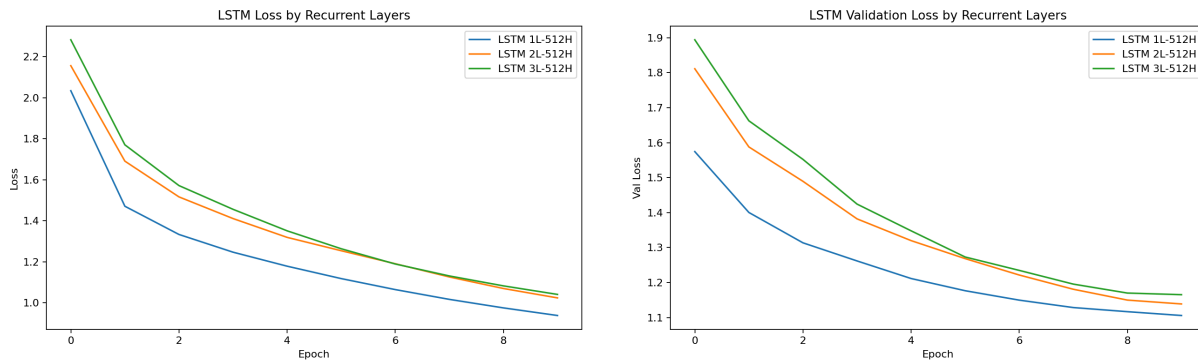
3.2.2 Perbandingan Jumlah *Layer* (LSTM)



Gambar 11. Nilai BLEU-4 dan METEOR pada Eksperimen Variasi *Recurrent Layer* LSTM

Gambar di atas menunjukkan perbandingan jumlah *layer* pada LSTM terhadap nilai BLEU-4, METEOR, disertai dengan grafik *loss* dan *validation loss* pada ilustrasi di bawah paragraf ini. Berdasarkan hasil eksperimen, ditemukan hasil yang berbeda dengan RNN. Performa terbaik dihasilkan oleh model dengan arsitektur 2 *layer*. Nilai ini menurun pada dua

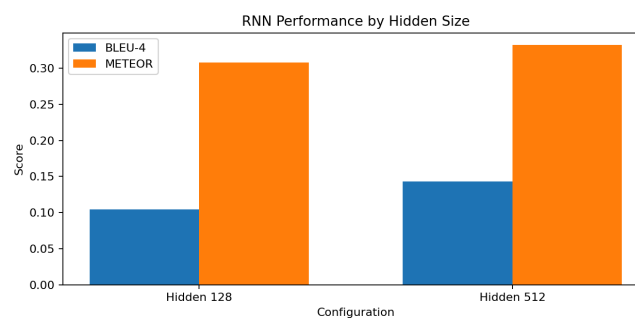
konfigurasi lainnya. Hal ini berarti bahwa, pada kasus ini, penambahan kompleksitas arsitektur dengan “cukup,” yaitu menjadi 2 *layer* masih meningkatkan performa model. Performa ini akan kembali menurun jika kompleksitas ini hilang (1 *layer*) atau ditambahkan secara berlebih (3 *layer*).



Gambar 12. Grafik *Loss* dan *Validation Loss* pada Eksperimen Variasi *Recurrent Layer* LSTM

Berbeda dengan kedua metrik di atas, berdasarkan grafik, model dengan 1 *layer* memberikan performa yang terbaik dan lebih stabil pada *validation loss*. Di sisi lain, kedua model lainnya memiliki pola *loss* yang sama, yaitu terus menurun. Namun, berbeda dengan RNN, pola *loss* LSTM bergerak dengan jauh lebih stabil. Hal ini terjadi berkat adanya *gate-gate* LSTM yang meminimalisasi dampak *vanishing gradient* yang menjadi salah satu masalah utama dalam RNN. Dengan *gate* tersebut, LSTM mampu melakukan generalisasi secara lebih baik ketimbang RNN.

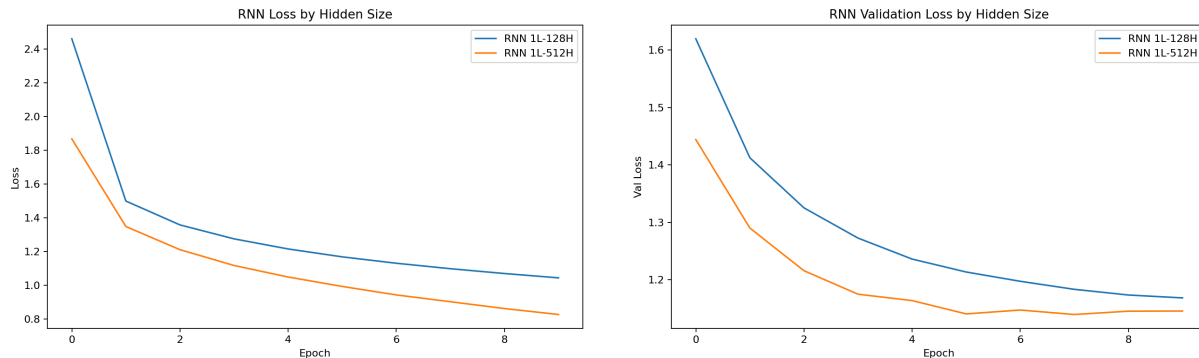
3.2.3 Perbandingan Ukuran *hidden state* (RNN)



Gambar 13. Nilai BLEU-4 dan METEOR pada Eksperimen Variasi *Hidden Size* RNN

Gambar di atas menunjukkan perbandingan *hidden size* pada RNN terhadap nilai BLEU-4, METEOR, disertai dengan grafik *loss* dan *validation loss* pada ilustrasi di bawah paragraf ini. Berdasarkan hasil eksperimen, peningkatan ukuran *hidden size* pada model RNN memberikan peningkatan performa pada kedua metrik. Hal ini dapat terjadi karena *hidden state*

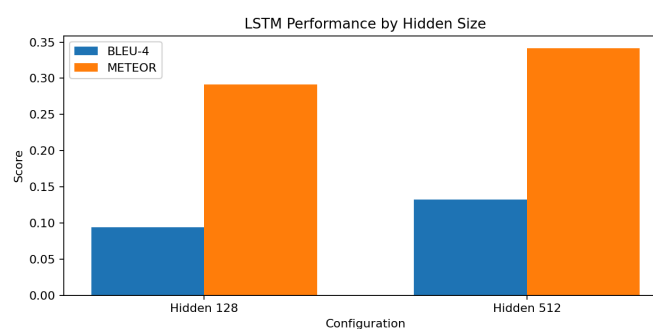
yang lebih besar memungkinkan model menyimpan representasi informasi yang lebih kaya dan kompleks selama proses pemodelan sekuens. Dengan ini, model lebih mampu memahami hubungan antar kata dan konteks untuk menghasilkan *caption* yang diharapkan lebih berkualitas.



Gambar 14. Grafik *Loss* dan *Validation Loss* pada Eksperimen Variasi *Hidden Size* RNN

Berdasarkan grafik, model dengan ukuran *hidden state* lebih tinggi menghasilkan pola *loss* dan *validation loss* yang relatif lebih baik. Artinya, peningkatan kompleksitas arsitektur masih mengarah pada peningkatan performa dan bukan ke arah sebaliknya. Hal yang menarik untuk diperhatikan adalah bahwa grafik *validation loss* pada *hidden size* 512 mulai bergerak sedikit tidak stabil pada beberapa *epoch* terakhir, sedangkan modelandingannya bergerak dengan stabil. Diperlukan analisis lebih lanjut untuk memastikan apakah ini merupakan indikasi *overfitting* pada model 512 atau bukan, berdasarkan ke mana *validation loss* bergerak jika *epoch* ditambahkan.

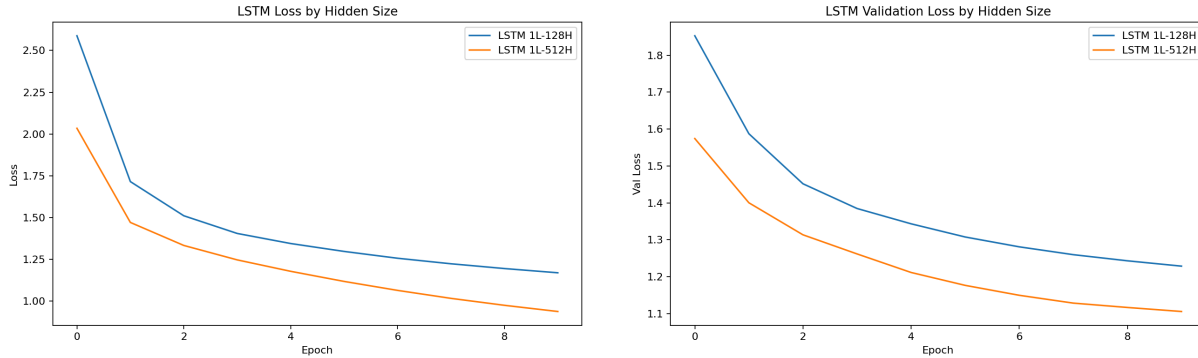
3.2.4 Perbandingan Ukuran *hidden state* (LSTM)



Gambar 15. Nilai BLEU-4 dan METEOR pada Eksperimen Variasi *Hidden Size* LSTM

Gambar di atas menunjukkan perbandingan *hidden size* pada LSTM terhadap nilai BLEU-4, METEOR, disertai dengan grafik *loss* dan *validation loss* pada ilustrasi di bawah paragraf ini. Berdasarkan hasil eksperimen, serupa dengan RNN, peningkatan ukuran *hidden size* pada model LSTM memberikan peningkatan performa pada kedua metrik. Hal ini dapat terjadi

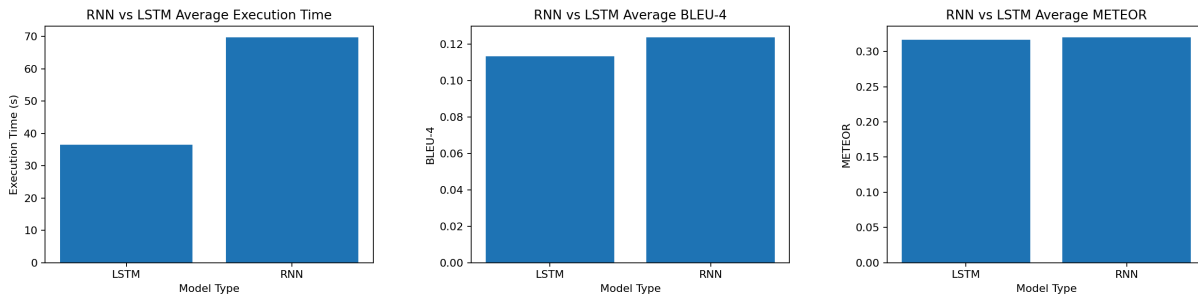
karena *hidden state* yang lebih besar memungkinkan model menyimpan representasi informasi yang lebih kaya dan kompleks selama proses pemodelan sekuens. Dengan ini, model lebih mampu memahami hubungan antar kata dan konteks untuk menghasilkan *caption* yang diharapkan lebih berkualitas.



Gambar 16. Grafik *Loss* dan *Validation Loss* pada Eksperimen Variasi *Recurrent Layer* LSTM

Berdasarkan grafik, model dengan kompleksitas lebih tinggi menghasilkan performa yang lebih baik. Hal yang menarik untuk diperhatikan adalah bahwa, tidak seperti RNN, grafik *loss* dan *validation loss* untuk kedua model LSTM bergerak menurun secara stabil tanpa fluktuasi nilai sedikitpun. Hal ini terjadi berkat adanya *gate-gate* LSTM yang meminimalisasi dampak *vanishing gradient* yang menjadi salah satu masalah utama dalam RNN. Dengan *gate* tersebut, LSTM mampu melakukan generalisasi secara lebih baik ketimbang RNN.

3.2.5 Perbandingan RNN vs LSTM

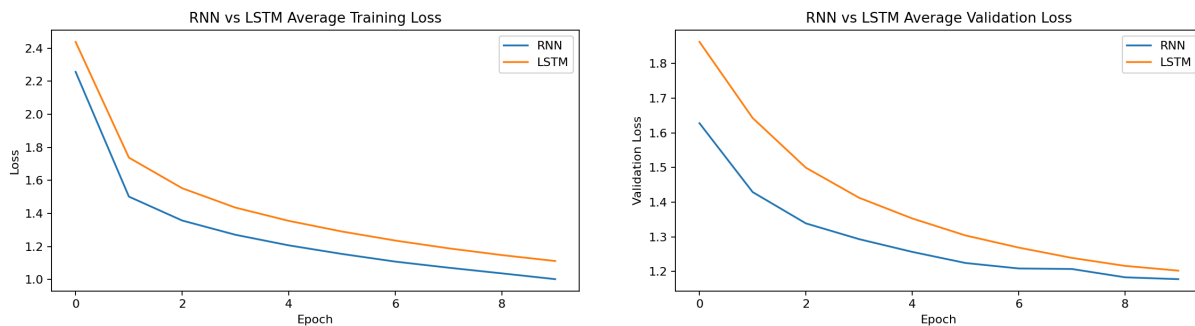


Gambar 17. Perbandingan Rata-Rata Waktu Eksekusi, Nilai BLEU-4, METEOR pada RNN dan LSTM

Gambar di atas menunjukkan rata-rata nilai waktu eksekusi, BLEU-4, dan METEOR, juga disertai dengan grafik *loss* dan *validation loss* pada ilustrasi di bawah paragraf ini, untuk membandingkan hasil implementasi model RNN dan LSTM. Berdasarkan hasil eksperimen, waktu eksekusi LSTM jauh lebih cepat ketimbang RNN. Secara logika, hal ini merupakan hasil yang tidak wajar. Kompleksitas LSTM jauh lebih tinggi ketimbang RNN untuk arsitektur yang sama, sehingga seharusnya RNN memerlukan waktu eksekusi yang lebih singkat. Perbedaan ini

terjadi karena perbedaan lingkungan eksperimen: *training* RNN dilakukan pada CPU sedangkan *training* LSTM dilakukan pada GPU yang mempercepat pemrosesan data dengan komputasi paralel.

Kemudian, berdasarkan metrik BLEU-4 dan METEOR, RNN menghasilkan performa yang lebih baik pada eksperimen ini. Hal ini dapat terjadi karena *dataset* yang diberikan tidak memerlukan keuntungan kompleksitas arsitektur pada LSTM. Artinya, bisa saja *task image captioning* pada konteks ini tidak memerlukan dependensi jangka panjang yang sangat kompleks untuk diatasi oleh LSTM, melainkan cukup dengan RNN. Artinya, meski secara teoritis LSTM meminimalisir masalah yang dimiliki RNN seperti dependensi jangka panjang dan *vanishing gradient*, dalam praktiknya tidak semua persoalan memerlukan kompleksitas tambahan tersebut untuk diselesaikan dengan baik.



Gambar 18. Perbandingan Grafik *Loss* dan *Validation Loss* pada RNN dan LSTM

Grafik di atas memperkuat argumen yang telah dijelaskan sebelumnya. Terlihat bahwa, berdasarkan *loss* dan *validation loss*, model RNN masih memiliki performa yang lebih baik ketimbang LSTM. Artinya, model ini mampu melakukan generalisasi secara lebih baik pada data/kasus yang diberikan. Namun, perlu diperhatikan bahwa terdapat sedikit ketidakstabilan pada *validation loss* RNN yang tidak ditemukan pada LSTM. Artinya, LSTM tetap bekerja untuk memastikan proses *training* berjalan stabil, meskipun hal tersebut tidak menjamin bahwa *training* akan menghasilkan model dengan performa lebih baik.

Tabel 6. Tabel Perbandingan *Caption* Hasil RNN dan LSTM

N o	Gambar	Caption RNN	Caption LSTM	Ground Truth
--------	--------	-------------	--------------	--------------

1		a man in a red jacket is standing in front of a large crowd of people	a man in a white shirt and a white hat is standing in front of a crowd of people	['a small dog walks with men in the city', 'some people waiting to cross a busy street in portland oregon', 'the people await to cross the street while walking the dog', 'three guys are waiting to cross the street one has a dog', 'three people wait to cross a city street and one has a dog on a leash']
2		a woman in a black shirt and tie is smiling	a man in a blue shirt and a white shirt is holding a cigarette	['a group of people wait in similar attire', 'an asian girl wearing a red hat along side several others with red hats as well', 'a small group of asian women wearing red head scarves and black shirts are shopping', 'five japanese ladies wear red bandanas', 'several asian women wearing red cloths on their heads']
3		a man is riding a horse on a racetrack	a man is riding a bike down a dirt road	['a hiker walks on rocky ground at the base of a foggy mountain', 'a lone hiker walking on dirt with a snowy mountain background', 'a person is hiking

				along rough terrain with fog and mountains in the background', 'distant person with blue backpack hiking in rocky area mountains in background', 'the person is hiking']
4		two dogs are playing in a field	two dogs are playing with a toy	['a big dog is biting a smaller dog on the leg', 'a dog bites the tail of another dog', 'a large brown dog sniffs a small white dog s behind', 'a large brown dog trying to catch a small white puppy from behind', 'the large brown dog is sniffing a small white dog']
5		a man in a wetsuit is surfing on a boat	a man is skiing down a snowy hill	['a man bracing himself against the pull of a large kite', 'a man holds his parachute', 'man trying to guide a kit in the desert', 'the man crouched down as he held onto the parachute that was caught in the breeze', 'the man slides in the sand while holding on to his hang glider']

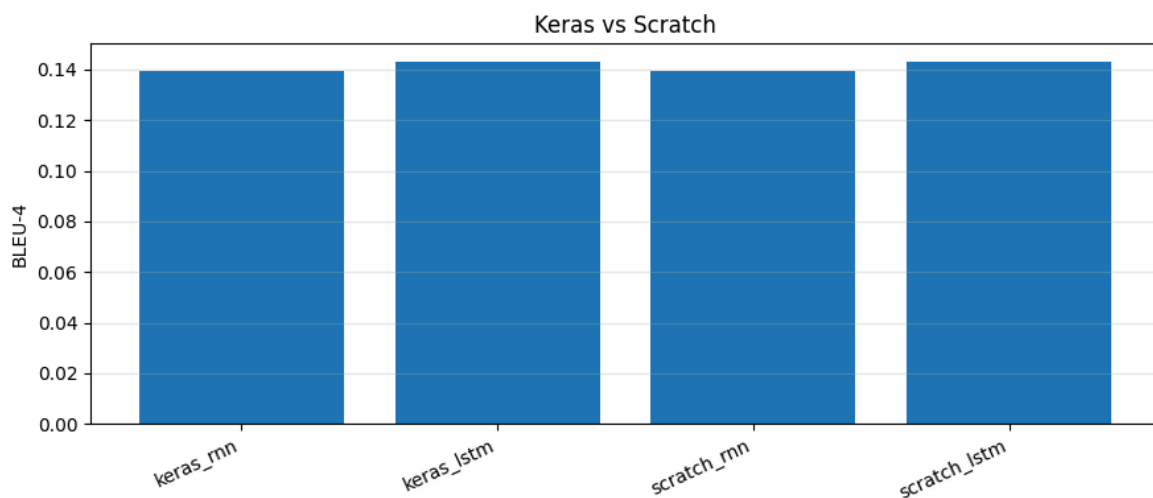
6		a boy is jumping in a puddle	a young boy jumps off a swing	['two girls playing at a playground', 'two girls playing on parallel bars at the playground', 'two girls play on a set of parallel bars', 'two little girls playing on the parallel bars', 'two young girls play on a set of parallel bars in a playground']
7		a girl in a pink shirt is running on a beach	a girl in a pink shirt and a white shirt is playing with a white ball in the sand	['a boy and girl playing with buckets in a lawn sprinkler', 'a girl and boy are playing with buckets of water in the garden overlooking a lake', 'a girl with a bucket of water is being chased by a boy with a bucket of water', 'a young girl and young boy are playing with pails of water with the ocean in the background', 'children play in a sprinkler near a beach']

8		a tennis player in a blue uniform is running through a field	a man in a blue shirt is playing with a soccer ball	['a boy in a white shirt and blue shirts is playing tennis', 'a boy swings his tennis racket at a tennis ball', 'a kid in a black cap and light blue shorts playing tennis', 'a person with sunglasses blue shorts and a white shirt hitting a tennis ball', 'a woman is hitting a tennis ball']
9		a man in a leather jacket and tie earrings	a man in a blue shirt and a white shirt is sitting on a bench	['a man holding a sign with people walking around in the background', 'a man in an old suit holding up a protest sign in a crowd of people', 'a man stands by an animal rights sign at an outdoor event', 'a man stands near a crowd holding a protest sign', 'man with tan cap suit jacket and plaid shirt behind a picture in an outdoor scene with other people in the background']
10		a man is surfing on a beach	a young boy is jumping off a slide into a pool	['a man is wind sailing in the ocean', 'a man stands on a sailboat in the water', 'a man windsurfs near the beach', 'a person

				is windsurfing in the water', 'a windsurfer in near the beach']
--	--	--	--	---

Analisis lebih lanjut yang dapat dilakukan adalah analisis kualitatif terhadap *caption* yang dibandingkan terhadap gambar yang bersangkutan juga dengan *caption ground truth* yang tersedia. Hal yang jelas tampak dari kedua arsitektur adalah bahwa keduanya mampu menghasilkan kalimat yang koheren dan mampu mengidentifikasi keberadaan manusia (*man, woman, boy, girl*), meski masih kurang akurat dalam mengidentifikasi jenis kelamin manusia tersebut. Jika dibandingkan antara RNN dengan LSTM, berdasarkan contoh yang ditampilkan, secara kualitatif RNN menghasilkan *caption* yang lebih baik. Ia mampu mengidentifikasi komponen-komponen lebih detail pada gambar seperti *black shirt* dan *smiling* pada gambar 2, *running on a beach* pada gambar 7, *a tennis player* pada gambar 8, serta *a man in a leather jacket* pada gambar 9

3.2.6 Perbandingan Keras vs from scratch

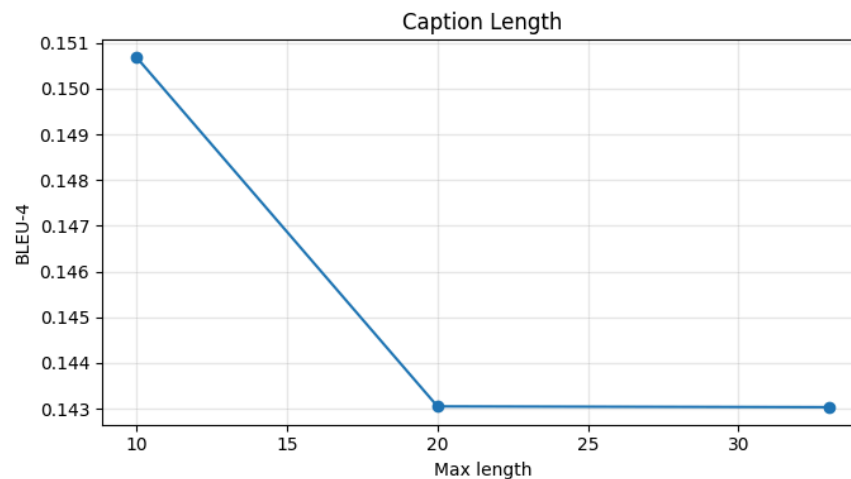


Gambar 19. Perbandingak Keras vs from scratch

Berdasarkan grafik, performa model hasil implementasi menggunakan Keras dan implementasi from scratch menunjukkan nilai BLEU-4 yang relatif berdekatan. Hal ini mengindikasikan bahwa implementasi from scratch telah mampu mereplikasi perilaku utama dari model berbasis Keras dengan cukup baik. Pada model RNN maupun LSTM, nilai BLEU-4 yang dihasilkan tidak mengalami perbedaan yang signifikan, sehingga dapat disimpulkan bahwa proses perhitungan pada implementasi manual sudah berjalan secara konsisten terhadap model pembandingnya.

Hal yang menarik untuk diperhatikan adalah bahwa model LSTM, baik pada Keras maupun from scratch, menghasilkan nilai BLEU-4 yang sedikit lebih tinggi dibandingkan model RNN. Hal ini selaras dengan karakteristik LSTM yang memiliki mekanisme gate untuk mengontrol aliran informasi, sehingga lebih mampu mempertahankan konteks dalam urutan kata yang lebih panjang. Dengan demikian, implementasi from scratch tidak hanya berhasil merepresentasikan struktur model, tetapi juga mampu menangkap pola performa yang sama seperti implementasi Keras.

3.2.7 Pengaruh panjang maksimum *caption* terhadap BLEU-4



Gambar 20. Grafik Hubungan Panjang Maksimum *Caption* dengan Nilai BLEU-4

Berdasarkan grafik, nilai BLEU-4 tertinggi diperoleh ketika panjang maksimum caption berada pada nilai 10. Setelah panjang maksimum dinaikkan menjadi 20 dan 33, nilai BLEU-4 mengalami penurunan dan cenderung stabil pada nilai yang lebih rendah. Hal ini menunjukkan bahwa penggunaan caption yang terlalu panjang tidak selalu meningkatkan kualitas prediksi model, terutama jika tambahan panjang tersebut membuat model harus mempelajari dependensi kata yang lebih kompleks.

Penurunan performa ini dapat terjadi karena semakin panjang urutan caption, semakin besar pula ruang kemungkinan kata yang harus diprediksi oleh model. Akibatnya, model menjadi lebih sulit mempertahankan ketepatan urutan kata secara konsisten. Pada metrik BLEU-4, kesesuaian urutan empat kata secara berurutan sangat berpengaruh, sehingga kesalahan kecil pada caption panjang dapat menurunkan skor secara signifikan. Dengan demikian, panjang maksimum caption yang lebih pendek dapat memberikan hasil yang lebih optimal karena model lebih fokus pada informasi utama dari gambar dan tidak terlalu terbebani oleh urutan kata yang panjang.

IV. Kesimpulan dan Saran

4.1 Kesimpulan

Berdasarkan hasil pengujian mendalam terhadap model yang diimplementasikan secara mandiri, berikut adalah poin-poin kesimpulan dari seluruh eksperimen:

1. Model CNN dalam eksperimen yang memberikan hasil terbaik adalah konfigurasi dengan 3 *layer*, 64 *filter*, 3 *kernel* dan dengan *max pooling* dengan *macro F1-score* sebesar 0.7607.
2. Beberapa variasi konfigurasi, misalnya jumlah *layer* konvolusi dan ukuran *filter* memberikan tren yang jelas terhadap performa model, sedangkan beberapa konfigurasi lain tidak menampilkan pola yang begitu jelas.
3. Secara umum, model RNN menghasilkan performa yang lebih baik ketimbang LSTM, baik secara kuantitatif atau kualitatif.
4. Model-model berbasis LSTM menghasilkan grafik *loss* dan *validation loss* yang relatif jauh lebih stabil ketimbang RNN.

4.2 Saran

Berdasarkan temuan di atas, beberapa hal yang dapat dikembangkan untuk penelitian atau tugas selanjutnya adalah:

1. Melakukan eksperimen dengan variasi arsitektur lebih banyak, dengan kompleksitas lebih tinggi
2. Melakukan eksplorasi terhadap penelitian lebih lanjut mengenai *encoder-decoder* dalam konteks *image captioning*
3. Melakukan eksplorasi terhadap permasalahan lain yang dapat diselesaikan oleh CNN, RNN, dan LSTM, baik secara terpisah maupun dalam arsitektur *encoder-decoder*

V. Pembagian Tugas

Berikut pembagian tugas untuk tugas besar ini.

Nama	NIM	Tugas
Muhammad Ghifary Komara Putra	13523066	Implementasi RNN, implementasi LSTM, penulisan laporan
Muh. Rusmin Nurwadin	13523068	Arsitektur model, eksperimen, penulisan laporan
Aryo Wisanggeni	13523100	Implementasi CNN, eksperimen, penulisan laporan

Referensi

- [1] Andrej Karpathy, “The spelled-out intro to neural networks and backpropagation: building micrograd,” youtube.com, August 17, 2022. Available: <https://www.youtube.com/watch?v=VMj-3S1tku0>. [Accessed Mar. 18, 2026]
- [2] Tanti et al., “Where to put the Image in an Image Caption Generator,” arxiv.com, March 14, 2018. Available: <https://arxiv.org/abs/1411.4555>. [Accessed May 15, 2026]
- [3] Vinyals et al., “Show and Tell: A Neural Image Caption Generator,” arxiv.com, April 20, 2015. Available: <https://arxiv.org/abs/1411.4555>. [Accessed May 15, 2026]

Lampiran

Lampiran A: Tautan menuju repositori Github

Tautan Github : https://github.com/Rusmn/ML-Tubes-2_RecursiveLearnaholic

Lampiran B: Form Penggunaan AI

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, saya/kami*:

Nama/NIM : Muhammad Ghifary Komara Putra, M Rusmin Nurwadin, Aryo
Wisanggeni/13523066, 13523068, 13523100
Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika
Kode Mata Kuliah : IF3270
Nama Mata Kuliah : Pembelajaran Mesin
Judul Tugas/Proposal : Tugas Besar 2 IF3270 Pembelajaran Mesin Convolutional Neural Network dan Recurrent Neural Network

Menyatakan bahwa saya/kami* menggunakan/~~tidak menggunakan~~ AI dalam pengerjaan maupun penyusunan tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Ya/ Tidak *	Membantu memperbaiki “typo” atau kesalahan penggunaan tanda baca.
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya/ Tidak *	Membantu parafrase penjelasan pada laporan
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya/ Tidak *	Membantu mempelajari dan mendalami konsep-konsep terkait pengerjaan tugas besar

Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Ya /Tidak*	Tidak digunakan
--	-----------------------	-----------------

Saya/Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 16 Mei 2026



Muhammad Ghifary Komara Putra
13523066



M Rusmin Nurwadin
13523068



Aryo Wisanggeni
13523100